Auditoría, Calidad y Fiabilidad Informáticas

(IT Auditing, Quality, and Reliability)

Universidad Complutense de Madrid Academic Year 2026/27

Final Project: Specification and Verification of a Real Architecture in Maude

---

This project is carried out in groups of two or three students. Each group will choose a real architecture, protocol, or application (*not* a toy example invented for the occasion), study it in depth, and specify it in Maude, proving properties about it.

Exercise 1 Choose a real system and study it thoroughly enough to be able to explain, in your own words, how it works. You may choose one from the list below, or propose a different one (justifying its interest, subject to the instructor's prior approval).

Each system is marked with the team size its scope is meant for: **[2]** fits a pair, **[3]** calls for a trio, and **[2–3]** works with either. The list runs from the most to the least demanding. This is a recommendation, not a restriction, and **choosing a system marked [2] does not cap your grade**: what is assessed is the depth of the development relative to the size of the team.

- **Simplified Raft-style consensus** (leader election plus log replication): there is at most one leader per term; committed entries are not lost when a minority of nodes fail. **[3]**
- **Two-phase commit (2PC)**: all participants decide the same outcome (atomicity); no indefinite blocking if the coordinator fails after the prepare phase. **[3]**
- **Distributed lock manager**: mutual exclusion between clients, no deadlock, the lock is released after its holder fails. **[3]**
- **Messaging system with delivery guarantees** (at-least-once or exactly-once): every sent message is delivered at least once; no duplicates when exact delivery is required. **[3]**
- **Cache with invalidation** (write-through or write-back): consistency between the cache and persistent storage, no stale reads after an invalidation. **[2–3]**
- **Handshake of a real protocol** (e.g., the TCP three-way handshake, or a simplified version of TLS): the handshake always ends in a consistent state on both ends. **[2–3]**
- **Leader election** (the bully algorithm or a similar one): a single leader is elected, and it is always the highest-priority node among those alive. **[2–3]**
- **Gossip protocol**: information eventually propagates to all live nodes; state convergence. **[2–3]**
- **Producer-consumer with a bounded queue**: no message loss, no buffer overflow, no consumer starvation. **[2]**
- **Rate limiter** (token bucket or leaky bucket): no more requests than the configured limit are ever accepted within a time window; the system recovers after a traffic spike. **[2]**
- **Circuit breaker** (a resilience pattern in microservices): the circuit opens after N consecutive failures, and no calls are made to the failed service while the circuit is open. **[2]**
- **Load balancer** (round robin or least connections): fair distribution of requests among active nodes; no failed node receives traffic. **[2]**

In the report you must explain which essential mechanisms of the real system you are going to model and, explicitly, what is simplified or left out and why that simplification does not undermine the properties you want to study. **The depth of the development of the chosen system will be taken into account in the grading**: a system with more concurrency, more edge cases, or properties that are harder to express and verify will be valued more highly.

Exercise 2 Specify the chosen system in Maude. Define the sorts and data structures you consider necessary, choose carefully whether something should be written as an equation or as a rule, and split the problem into modules (functional modules for the domain's data types, a system module for the protocol's dynamics). Comment your code and include clear instructions for loading it.

Also write at least one example term with a reasonable number of agents/nodes/messages in the system (justify why this example is sufficiently representative, and not a degenerate case).

Exercise 3 Using what you defined in part 2, define at least two relevant properties of the system (for example: mutual exclusion, absence of deadlock, in-order delivery, no message loss, termination, absence of starvation). Avoid trivial properties that say nothing meaningful about the system.

Exercise 4 Check whether the properties from part 3 hold:

- Use the `search` command to explore the state space and check (or refute) the reachability of undesired states.
- Formulate at least one temporal (LTL) property and check it with Maude's model checker, on a module that includes `MODEL-CHECKER`.

Include the actual Maude output in the report (a literal transcript of the commands and their results, not just a description of what "should" happen), and briefly explain the model checker's result in each case.

Exercise 5 Deliberately introduce, on your own specification, at least **two mutations** that break one of the properties you verified in part 4 (for example: removing a guard condition, swapping the order of two rules, dropping a case from a pattern). Re-run `search` or the model checker on each mutant and show that the property now fails, including the counterexample or offending state found by Maude.

Explain, in terms of the real system (not just the Maude code), what concrete error each mutation represents: for example, "this simulates what would happen if the holder of a lock fails without releasing it."

Exercise 6 Write which rules could be turned into equations to optimize the model checking of each property from part 4 (it is enough to name the rules to change; you do not need to modify the code).

Submission. In addition to the Maude code, submit a report that integrates the parts above into a coherent document. **The project will be graded with Maude 3.5**: you may work with any version you like, but check before submitting that your specification loads without warnings and gives the same results in that one.

Include in the report a **task split**: a table saying, for each of parts 1–6, who led it, plus an approximate contribution percentage per member adding up to 100. The split is agreed and signed by the whole team. It need not be even: an uneven split declared honestly is not penalized, and a cosmetic one shows up in the defense.

Defense. Each team will defend its project orally during class hours, with individual questions to each member about specific modeling decisions, the properties verified, the mutations from part 5, and possible extensions. In addition, each member will be asked for a **small modification to your own specification**. **The defense carries no marks of its own**, but it is a **necessary condition for passing the project**: a student who does not pass it does not pass the project, however good the submitted work, and therefore loses the share of the course grade that this block carries.

On the use of AI assistants. Using AI is permitted, as long as it is instrumental. If you do use it, the MCP for Maude is recommended.

Grading. The project grade is made up of:

Maude specification (structure, equations vs. rules, comments, example term)	25%
Properties defined (part 3)	10%
Verification: `search` and the LTL model checker (part 4)	25%
Mutations (part 5)	20%
Rules-to-equations optimization	5%
Report (coherence, justification of simplifications, depth of the development)	15%

This is a team grade, weighted by the declared task split, which the instructor may adjust if the defense does not support it. The defense is not in the table because it adds no percentage: you have to pass it to pass the project, and it is assessed per student, so a defense revealing that a member does not understand what they submitted can place that student below the rest of the team, or fail them outright, independently of the breakdown above.